

Environment Setup & Installation

1. Prerequisites

- Linux OS (Ubuntu 20.04+ recommended)
- CMake 3.16+
- OpenSSL 1.1.1+ (for certificate generation)
- G++ / GCC compiler supporting C++17

2. Building Microsoft MsQuic

Cloning and building MsQuic from source:

```
git clone --recurse-submodules https://github.com/microsoft/msquic.git
cd msquic

# Configure and compile MsQuic in Debug/Logging mode
cmake -S . -B build \
  -DCMAKE_BUILD_TYPE=Debug \
  -DQUIC_ENABLE_LOGGING=ON \
  -DQUIC_BUILD_TOOLS=ON

cmake --build build -j$(nproc)
```

Ensure the library path matches the CMake configuration. By default, CMake targets `$ENV{HOME}/msquic` and links against `${MSQUIC_ROOT}/build/bin/Debug/libmsquic.so`.

3. Generating TLS Certificates

The QUIC protocol mandates encrypted connections, therefore the server needs a key and credential pair. We can use OpenSSL to generate self-signed test credentials.

It is recommended to create a `certs/` folder in the project root directory to store these credentials.

```
mkdir -p certs
cd certs

openssl req -x509 -newkey rsa:2048 \
  -keyout server.key \
  -out server.crt \
  -days 365 \
  -nodes \
  -subj "/CN=localhost"
```

4. Test msquic with quicsample

Before setting up this project, you can use MsQuic's built-in testing program to ensure that your QUIC environment and credentials are working correctly.

Start the test server:

```
cd msquic/build

./quicsample \
  -server \
  -cert_file:~/certs/server.crt \
  -key_file:~/certs/server.key
```

Start the test client:

```
cd msquic/build

./quicsample \
  -client \
  -target:127.0.0.1 \
  -unsecure
```

Build and Run QUIC Media System

Configure and compile the client and server applications:

```
# From project root directory
mkdir -p build
cd build
cmake ..
make -j$(nproc)
```

This builds two executables in the `build/` directory:

- `moq_server` : The listening media sink server.
- `moq_client` : The file parsing and media streaming client.

Usage Guide

1. Run the Server

Start the server listener first to receive the streamed output:

```
./moq_server <host> <port> <output.h265> <cert_file> <key_file>
```

- **Example:**

```
./moq_server 0.0.0.0 4433 output.h265 ../certs/server.crt ../certs/server.key
```

2. Run the Client

Stream the input H.265 file to the server:

```
./moq_client <host> <port> <input.h265> [num_streams] [chunk_size] [min_mtu] [max_mtu]
```

- `num_streams` (Optional): The total number of streams to open (Default: `2`, minimum `2` to split priority flows).
- `chunk_size` (Optional): Payload fragment size limit in bytes (Default: `1200`).
- `min_mtu` / `max_mtu` (Optional): Forces MSQuic to override PMTUD and constrain the wire packet sizes. (Note: Must satisfy `max_mtu >= min_mtu >= 1200` due to MsQuic limits; invalid values will cause initialization to fail).
- **Example:**

```
./moq_client localhost 4433 4 test_video.h265 1000 1200 1200
```